2026-2027 学年第一学期

《高级语言程序设计(C++语言)》

实验报告

实验名称：	函数的使用与 IDE 调试（实验二）		
实验日期：	2026.10.30	实验地点：	于刚宋晓楼 A237
学　　号：	2025302183225	姓　　名：	徐维鑫
学　　院：	人工智能学院	专　　业：	人工智能专业
指导老师：	丁建立		

一、实验目的

1. 掌握函数的定义、声明与调用方法；

2. 掌握值传递、引用传递、数组参数和 const 参数的使用；

3. 掌握默认参数、函数重载等函数特性；

4. 理解递归的思想，能编写并分析递归函数；

5. 理解局部变量、静态局部变量和全局变量的作用域与生存期；

6. 掌握 VSCode + gdb 的调试方法：设置断点、单步执行、逐过程/逐语句、监视变量；

7. 养成先分析、再编码、编译开警告、运行后验证的习惯。

二、实验环境

操作系统：Windows 11（版本 10.0.26200）。

编译器:MinGW-w64 GCC 8.1.0(g++),编译选项 -std=c++11 -Wall -Wextra。

编辑器与调试器：Visual Studio Code + C/C++ 扩展，GNU gdb 8.1。

项目组织：每个题目单独建立一个 .cpp 源文件，单独编译为可执行文件后运行；仓库根目录的 .vscode 保存构建与调试配置。

三、实验过程及结果

本次实验按照函数基础、参数传递、递归与作用域、IDE 调试四组展开，共完成 15 个程序。每个题目单独建立一个源文件并单独编译运行，全部源码位于仓库 lab02/src/ 目录，下面按练习顺序给出各题目的代码、运行结果与分析。

练习一　函数的定义、声明与调用

1. 计算圆柱体积

实验内容：把圆柱体积的计算封装为 cylinderVolume 函数，函数声明写在 main 之前、定义写在 main 之后，输入半径和高后输出体积。

代码：

```
#include <iostream>
#include <iomanip>
using namespace std;

const double PI = 3.1415926;

double cylinderVolume(double radius, double height);  // 函数声明

int main() {
    double radius = 0.0, height = 0.0;
    cout << "请输入圆柱的半径和高：" << endl;
    cin >> radius >> height;
    if (radius <= 0 || height <= 0) {
        cout << "半径和高必须为正数。" << endl;
        return 1;
    }
    cout << fixed << setprecision(2);
    cout << "圆柱体积：" << cylinderVolume(radius, height) << endl;
    return 0;
}

double cylinderVolume(double radius, double height) {  // 函数定义
    return PI * radius * radius * height;
}
```



图 1-1　计算圆柱体积运行结果

结果分析：输入半径 2.5、高 8，输出 157.08，与 $\pi \times 2.5^2 \times 8$ 的手算结果一致。函数声明让 main 在函数定义之前就能调用它；radius、height 是值传递，函数内部不会修改实参。

2. 求最大公约数

实验内容：用辗转相除法把求最大公约数的过程封装成 gcd 函数，并测试正数、互质数、含 0 和负数的情况。

代码：

```
#include <iostream>
using namespace std;

int gcd(int a, int b) {
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main() {
    int data[5][2] = {{12, 18}, {100, 75}, {17, 5}, {0, 9}, {-24, 36}};
    for (int i = 0; i < 5; ++i) {
        cout << "gcd(" << data[i][0] << ", " << data[i][1] << ") = "
             << gcd(data[i][0], data[i][1]) << endl;
    }
    return 0;
}
```



图 1-2 求最大公约数运行结果

结果分析：gcd(12,18)=6、gcd(100,75)=25、gcd(17,5)=1、gcd(0,9)=9、gcd(-24,36)=12。循环在 b==0 时结束，此时 a 就是最大公约数；函数开头先取绝对值，使负数输入也能得到正确结果。

3. 输出 1~100 的素数

实验内容：定义返回 bool 的 isPrime 函数，在 main 中遍历 1~100，统计并输出全部素数。

代码：

```
#include <iostream>
using namespace std;

bool isPrime(int n);  // 函数声明

int main() {
    int count = 0;
    cout << "1~100 内的素数：" << endl;
    for (int i = 2; i <= 100; ++i) {
        if (isPrime(i)) {
            cout << i << " ";
            ++count;
        }
    }
    cout << endl << "共 " << count << " 个。" << endl;
    return 0;
}

bool isPrime(int n) {  // 函数定义
    if (n < 2) return false;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) return false;
    }
    return true;
}
```

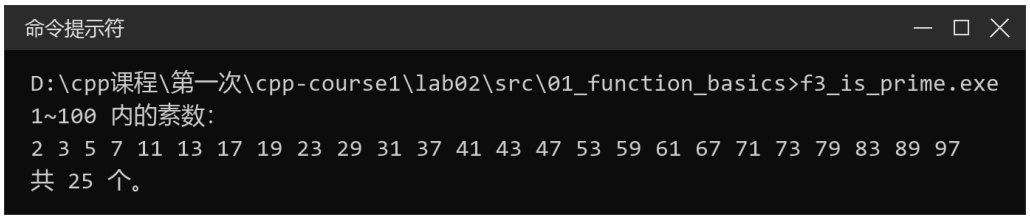


图 1-3 输出 1~100 的素数运行结果

结果分析：共输出 25 个素数。循环条件 i*i<=n 只试除到 √n，比试除到 n-1 快得多；n<2 直接返回 false，所以 1 不会被误判为素数。bool 返回值可以直接写在 if 条件中。

4. 输出菱形

实验内容：用 printChars 完成“连续输出 count 个字符”，再用 printDiamond 分上下两部分控制空格和星号。

代码：

```
#include <iostream>
using namespace std;

void printChars(int count, char ch) {
    for (int i = 0; i < count; ++i) cout << ch;
}

void printDiamond(int halfHeight) {
    for (int i = 1; i <= halfHeight; ++i) {
        printChars(halfHeight - i, ' ');
        printChars(2 * i - 1, '*');
        cout << endl;
    }
    for (int i = halfHeight - 1; i >= 1; --i) {
        printChars(halfHeight - i, ' ');
        printChars(2 * i - 1, '*');
        cout << endl;
    }
}

int main() {
    int n = 0;
    cout << "请输入菱形的半高（1~9）: " << endl;
    cin >> n;
    if (n < 1 || n > 9) {
        cout << "输入超出范围。" << endl;
        return 1;
    }
    printDiamond(n);
    return 0;
}
```



图 1-4 输出菱形运行结果

结果分析：半高为 4 时输出 7 行、共 16 个星号。printChars 把重复输出的循环封装起来，printDiamond 只负责计算每行的空格数和星号数；函数嵌套调用后，main 中只剩一行 printDiamond(n)。

练习二　参数传递与函数进阶

1. 值传递与引用传递

实验内容：分别用值传递和引用传递实现交换函数，比较调用前后实参的值。

代码：

```
#include <iostream>
using namespace std;

void swapByValue(int a, int b) {
    int t = a;
    a = b;
    b = t;
    cout << "值传递函数内部：a = " << a << ", b = " << b << endl;
}

void swapByReference(int &a, int &b) {
    int t = a;
    a = b;
    b = t;
    cout << "引用传递函数内部：a = " << a << ", b = " << b << endl;
}

int main() {
    int x = 3, y = 8;
    cout << "调用前：x = " << x << ", y = " << y << endl;
    swapByValue(x, y);
    cout << "值传递调用后：x = " << x << ", y = " << y << endl;
    swapByReference(x, y);
    cout << "引用传递调用后：x = " << x << ", y = " << y << endl;
    return 0;
}
```



图 2-1　值传递与引用传递运行结果

结果分析：值传递函数内部 a、b 变成 8、3，但调用后 x、y 仍是 3、8；引用传递调用后 x、y 变为 8、3。值传递交换的只是形参副本，引用传递的形参是实参的别名，能真正修改实参。

2. 默认参数

实验内容：声明 power(base, exp=2)，分别用默认参数和显式参数调用。

代码：

```
#include <iostream>
using namespace std;

double power(double base, int exp = 2);  // 默认参数写在声明中

int main() {
    cout << "power(5)      = " << power(5) << endl;
    cout << "power(2, 10)  = " << power(2, 10) << endl;
    cout << "power(1.5, 3) = " << power(1.5, 3) << endl;
    return 0;
}

double power(double base, int exp) {  // 定义中不再重复默认值
    double result = 1.0;
    for (int i = 0; i < exp; ++i) result *= base;
    return result;
}
```



图 2-2 默认参数运行结果

结果分析：power(5)=25（exp 取默认值 2）、power(2,10)=1024、power(1.5,3)=3.375。默认参数只在声明中写一次，定义中重复写会报“默认参数重定义”；默认参数还必须从右向左连续设置。

3. 函数重载

实验内容：定义三个同名 maxOf 函数，分别处理两个 int、两个 double 和三个 int。

代码：

```
#include <iostream>
using namespace std;

int maxOf(int a, int b) {
    return a > b ? a : b;
}

double maxOf(double a, double b) {
    return a > b ? a : b;
}

int maxOf(int a, int b, int c) {
    int m = maxOf(a, b);
    return maxOf(m, c);
}

int main() {
    cout << "maxOf(3, 8)     = " << maxOf(3, 8) << endl;
    cout << "maxOf(3.5, 2.8) = " << maxOf(3.5, 2.8) << endl;
    cout << "maxOf(4, 9, 6)  = " << maxOf(4, 9, 6) << endl;
    return 0;
}
```



图 2-3 函数重载运行结果

结果分析：编译器根据实参的类型和个数选择版本：maxOf(3,8)=8、maxOf(3.5,2.8)=3.5、maxOf(4,9,6)=9。函数重载只看参数列表，返回值类型不同不能构成重载。

4. 数组参数与 const

实验内容：把数组和长度传给 average、maximum 函数，数组参数用 const 修饰。

代码：

```
#include <iostream>
#include <iomanip>
using namespace std;

double average(const double scores[], int n) {
    double sum = 0.0;
    for (int i = 0; i < n; ++i) sum += scores[i];
    return sum / n;
}

double maximum(const double scores[], int n) {
    double m = scores[0];
    for (int i = 1; i < n; ++i) {
        if (scores[i] > m) m = scores[i];
    }
    return m;
}

int main() {
    double scores[] = {88.5, 92.0, 79.0, 95.5, 86.0, 91.0};
    int n = sizeof(scores) / sizeof(scores[0]);
    cout << fixed << setprecision(2);
    cout << "平均分：" << average(scores, n) << endl;
    cout << "最高分：" << maximum(scores, n) << endl;
    return 0;
}
```



图 2-4 数组参数与 const 运行结果

结果分析：平均分 88.67，最高分 95.50。数组参数会退化为指针，函数内部用 sizeof 得不到数组长度，所以必须另传 n；const 修饰保证函数不会修改原数组。

练习三　递归与变量作用域

1. 递归求阶乘

实验内容：用递归函数计算 0!~20!，观察结果与数据类型的取值范围。

代码：

```
#include <iostream>
using namespace std;

unsigned long long factorial(int n) {
    if (n < 0) return 0;
    if (n <= 1) return 1;
    return static_cast<unsigned long long>(n) * factorial(n - 1);
}

int main() {
    for (int n = 0; n <= 20; ++n) {
        cout << n << "! = " << factorial(n) << endl;
    }
    return 0;
}
```

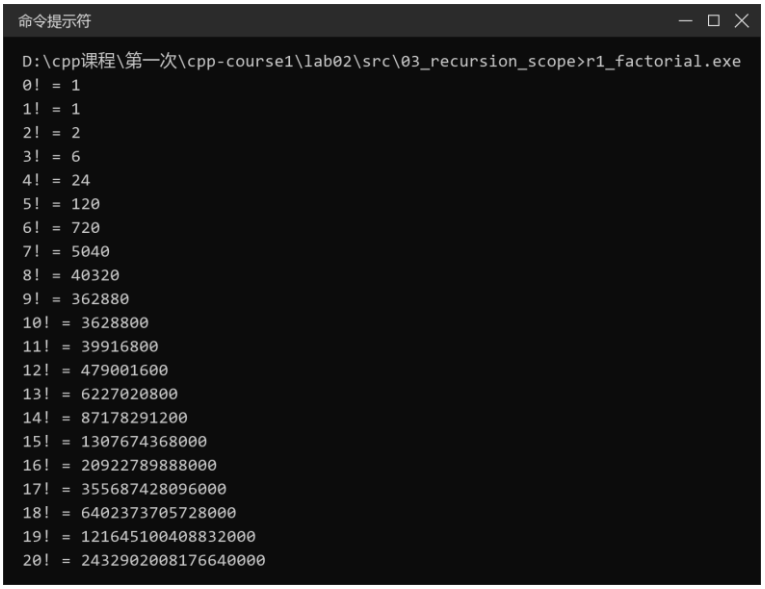


图 3-1　递归求阶乘运行结果

结果分析：0!~20! 全部正确，20!=2432902008176640000，仍在 unsigned long long 的范围内。递归终止条件 n<=1 返回 1，递归表达式 n×factorial(n-1) 让问题规模每次减 1，最终到达终止条件。

2. 斐波那契数列

实验内容：用递归和迭代两种方法求斐波那契数列前 20 项，对比结果与效率。

代码：

```
#include <iostream>
using namespace std;

long long fibRecursive(int n) {
    if (n <= 2) return 1;
    return fibRecursive(n - 1) + fibRecursive(n - 2);
}

long long fibIterative(int n) {
    if (n <= 2) return 1;
    long long a = 1, b = 1, c = 1;
    for (int i = 3; i <= n; ++i) {
        c = a + b;
        a = b;
        b = c;
    }
    return c;
}

int main() {
    cout << "n\t 递归\t 迭代" << endl;
    for (int n = 1; n <= 20; ++n) {
        cout << n << "\t" << fibRecursive(n) << "\t" << fibIterative(n) << endl;
    }
    return 0;
}
```

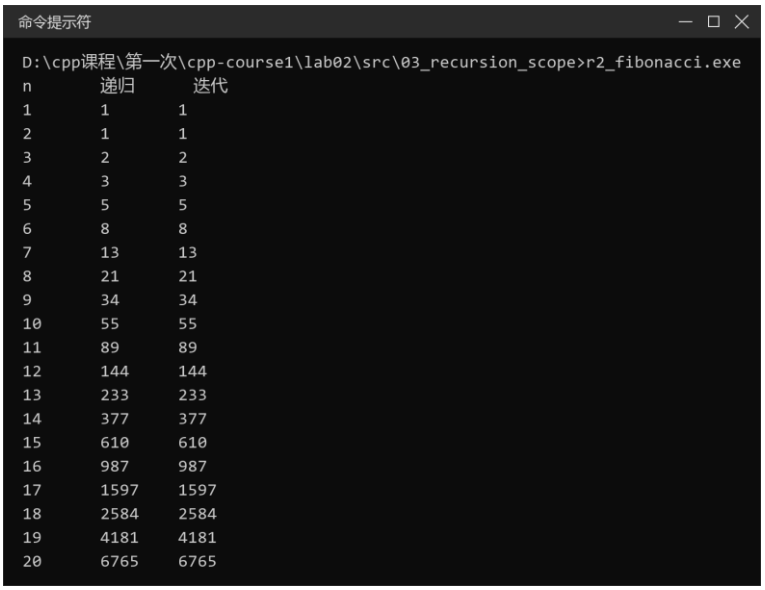


图 3-2 斐波那契数列运行结果

结果分析：两种方法结果完全一致，F(20)=6765。递归写法与定义式直接对应，但 F(n) 会重复计算大量子问题，时间复杂度约 O(2^n)；迭代只保留前两项，时间复杂度 O(n)，n 较大时应选迭代。

3. 汉诺塔

实验内容：用递归函数输出 3 个圆盘从 A 柱移动到 C 柱的每一步，并统计总步数。

代码：

```
#include <iostream>
using namespace std;

int moveCount = 0;  // 全局变量记录移动次数

void hanoi(int n, char from, char via, char to) {
    if (n == 1) {
        ++moveCount;
        cout << "第 " << moveCount << " 步: " << from << " -> " << to << endl;
        return;
    }
    hanoi(n - 1, from, to, via);
    ++moveCount;
    cout << "第 " << moveCount << " 步: " << from << " -> " << to << endl;
    hanoi(n - 1, via, from, to);
}

int main() {
    int n = 0;
    cout << "请输入圆盘个数（1~6）: " << endl;
    cin >> n;
    if (n < 1 || n > 6) {
        cout << "输入超出范围。" << endl;
        return 1;
    }
    hanoi(n, 'A', 'B', 'C');
    cout << "共移动 " << moveCount << " 次。" << endl;
    return 0;
}
```

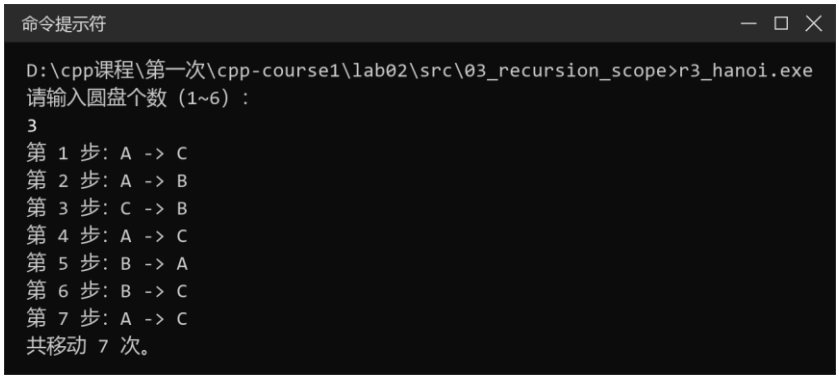


图 3-3 汉诺塔运行结果

结果分析：共移动 7 次，等于 2^3-1。hanoi 把“移动 n 个盘”分解为“把上面 n−1 个移到辅助柱、移动最大的盘、再把 n−1 个移到目标柱”三个子问题；全局变量 moveCount 记录步数。

4. 静态局部变量

实验内容：连续调用 countCall 三次，比较普通局部变量和 static 局部变量的输出。

代码：

```
#include <iostream>
using namespace std;

void countCall() {
    int normal = 0;        // 普通局部变量，每次调用重新初始化
    static int total = 0;  // 静态局部变量，只初始化一次
    ++normal;
    ++total;
    cout << "普通局部变量：" << normal << "，静态局部变量：" << total << endl;
}

int main() {
    for (int i = 0; i < 3; ++i) {
        countCall();
    }
    return 0;
}
```



图 3-4 静态局部变量运行结果

结果分析：普通局部变量每次都是 1，静态局部变量依次为 1、2、3。static 局部变量只在第一次调用时初始化，函数返回后仍然存在；它的作用域仍限于函数内部，但生存期贯穿整个程序。

练习四 IDE 调试

1. 算术运算示例（课件 DebugSample）

实验内容：用 Calculator 类的成员函数完成加、减、乘、除，演示断点、逐语句进入函数和监视变量。

代码：

```
#include <iostream>
using namespace std;

class Calculator {
public:
    double Calculate(double x, char oper, double y);
};

double Calculator::Calculate(double x, char oper, double y) {
    switch (oper) {
    case '+': return x + y;
    case '-': return x - y;
    case '*': return x * y;
    case '/': return x / y;
    default: return 0.0;
    }
}

int main() {
    double x = 0.0, y = 0.0;
    double result = 0.0;
    char oper = '+';
    cout << "算术运算示例程序" << endl << endl;
    Calculator c;
    while (oper != '$') {
        cout << "请输入运算式，格式如： a + b | a - b | a*b | a / b" << endl;
        cin >> x >> oper >> y;
        if (oper == '/' && y == 0) {
            cout << "Division by 0 exception" << endl;
            continue;
        } else {
            result = c.Calculate(x, oper, y);
        }
        cout << "Result is: " << result << endl;
    }
    return 0;
}
```

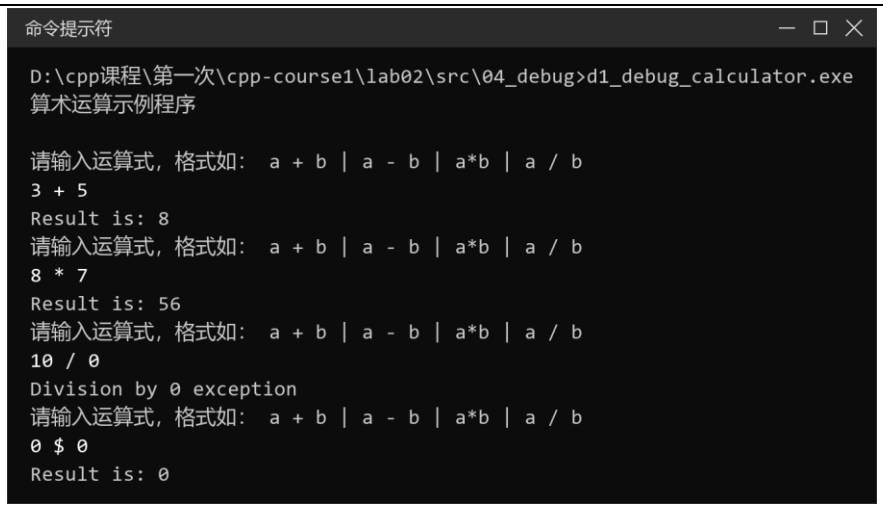


图 4-1 算术运算示例（课件 DebugSample）运行结果

结果分析：输入 3+5、8*7 分别输出 8、56；10/0 触发除零保护，输出 Division by 0 exception；输入 0 $ 0 后循环结束。在 result = c.Calculate(...) 处设断点，按 F11 可进入成员函数，观察 x、oper、y 和返回值。

2. 平均分程序（含逻辑错误）

实验内容：求 5 个学生成绩的平均分，程序中有两处编译器无法发现的逻辑错误。

代码：

```
#include <iostream>
using namespace std;

// 求数组平均分：本程序有两处逻辑错误，用于调试练习
double average(int scores[], int n) {
    int sum = 0;
    for (int i = 0; i < n - 1; ++i) {  // 错误 1：少加最后一个元素
        sum += scores[i];
    }
    return sum / n;                    // 错误 2：整数除法截断小数
}

int main() {
    int scores[] = {88, 92, 79, 95, 87};
    int n = sizeof(scores) / sizeof(scores[0]);
    cout << "平均分：" << average(scores, n) << endl;
    return 0;
}
```



图 4-2 平均分程序（含逻辑错误）运行结果

结果分析：程序输出 70，而手算的 5 个数的平均分应为 88.2。在 sum += scores[i] 处设断点并逐次按 F10，可看到 i 只到 3，循环条件 i < n - 1 少加了最后一个元素；把循环改对后仍输出 70，说明 sum / n 是整数除法，小数被截断。

3. 平均分程序（修正版）

实验内容：修正循环边界，并在返回前把 sum 转换为 double，再与原程序对照。

代码：

```
#include <iostream>
using namespace std;

// 修正后的平均分函数
double average(int scores[], int n) {
    int sum = 0;
    for (int i = 0; i < n; ++i) {        // 修正 1：遍历全部 n 个元素
        sum += scores[i];
    }
    return static_cast<double>(sum) / n;  // 修正 2：先转换为 double 再相除
}

int main() {
    int scores[] = {88, 92, 79, 95, 87};
    int n = sizeof(scores) / sizeof(scores[0]);
    cout << "平均分：" << average(scores, n) << endl;
    return 0;
}
```



图 4-3 平均分程序（修正版）运行结果

结果分析：输出 88.2，与手算一致。修正点有两个：循环条件改为 i < n，遍历全部元素；返回前用 static_cast<double>(sum) / n 做浮点除法。两处错误编译器都不会报错，只能靠断点、监视变量和对照预期结果发现。

四、实验总结（本次实验的收获、体会、经验和教训）

本次实验围绕函数的使用和 IDE 调试展开，完成了函数基础、参数传递、递归与作用域、调试练习四组共 15 个程序。所有源文件均用 g++ -std=c++11 -Wall -Wextra 编译通过并实际运行，运行结果保存在 lab02/screenshots 中。

函数基础部分让我理解了“声明、定义、调用”三者的关系：声明告诉编译器函数的名称、参数和返回值，定义给出具体实现，调用处只需要看到声明。把圆柱体积、最大公约数、素数判断、菱形输出分别封装成函数后，main 变得很短，逻辑也更清楚。const 常量、bool 返回值和 void 函数各自适合不同的场景。

参数传递是本次实验的重点。值传递交换不了实参，引用传递可以；数组参数会退化为指针，必须另传长度；const 数组参数能防止误修改；默认参数只能从右向左设置，且只在声明中写一次；函数重载依据参数列表而不是返回值。这些规则单看文字容易混淆，亲手写一遍、跑一遍就清楚多了。

递归部分最大的体会是“代码短不等于效率高”。阶乘和汉诺塔的递归写法与数学定义几乎一一对应，但斐波那契的递归会重复计算大量子问题，n=40 时就已经明显变慢，迭代写法只保留前两项，效率高得多。静态局部变量则让我直观看到了“作用域”和“生存期”不是一回事。

调试部分第一次真正用上了断点和单步。以前程序结果不对只能反复改代码、加 cout，现在可以在断点处停下来，用 F10 逐过程、F11 逐语句，在“变量”和“监视”面板里逐个核对。平均分程序的两处错误——循环少算一个元素、整数除法截断——编译器都不会报错，正是靠断点观察 i、sum、n 才定位到的。

本次实验的不足是：编写递归函数时曾漏写终止条件，导致程序栈溢出；部分函数的参数命名不够直观，降低了可读性。后续会继续做到：函数职责单一、参数命名清晰、编译时打开警告、写完先用小数据手工验算，再用调试器检查中间变量。